Source2Synth: Synthetic Data Generation and Curation Grounded in Real Data Sources

Alisia Lupidi[1,2], **Carlos Gemmell**[1,†], **Nicola Cancedda**[1], **Jane Dwivedi-Yu**[1], **Jason Weston**[1], **Jakob Foerster**[1,2], **Roberta Raileanu**[1], **Maria Lomeli**[1]

[1]Meta Superintelligence labs, [2]University of Oxford
[†]Work done during internship at Meta

Synthetic data generation has recently emerged as a promising approach for enhancing the capabilities of large language models (LLMs) without the need for expensive human annotations. However, existing methods often generate data that can be low quality or contrived. In this paper, we introduce *Source2Synth*, a scalable approach for synthetic data generation and curation that is grounded in real-world data sources. *Source2Synth* takes as input a custom data source and produces synthetic data examples with intermediate reasoning steps. Our method improves the dataset quality by discarding low-quality generations based on their answerability. We demonstrate the generality of this approach by applying it to two tasks that leverage two different types of data: multi-hop question answering (MHQA), where we test complex reasoning abilities leveraging documents, and tabular question answering (TQA), where we test tool usage leveraging tables. Our method improves performance by 25.51% for TQA on WikiSQL and 22.57% for MHQA on HotpotQA compared to the fine-tuned baselines.

Date: August 21, 2025
Correspondence: First and Last Authors: alisia@meta.com,marialomeli@meta.com

Meta

arXiv:2409.08239v2 [cs.CL] 20 Aug 2025

1 Introduction

Large Language Models (LLMs) (Devlin et al., 2019; Chowdhery et al., 2022; Brown et al., 2020; Vaswani et al., 2017) have risen in popularity due to their remarkable ability to process and generate human-like text (Radford et al., 2018). However, the scarcity of annotated task-specific data makes it difficult to unlock new capabilities, like using tools, and to enable LLMs to solve complex tasks, e.g. tasks that require multi-step reasoning.

One solution, consisting of enriching the data with human annotations collected for specific tasks, is expensive, time-consuming (Gilardi et al., 2023; Touvron et al., 2023), and prone to human errors and biases (Sylolypavan et al., 2023).

Alternatively, synthetic data that mimic real-world patterns can be constructed for fine-tuning, but ensuring the factuality and fidelity of the generated data remains challenging (Liu et al., 2024). Furthermore, constructing synthetic data for complex tasks that involve multi-step reasoning or that require aggregating complex information, for example, leveraging a table with SQL as the intermediate step for answering a question, is an open research question. If a general recipe for creating instruction-tuning synthetic data is used, only a small fraction of synthetic examples are suitable for multi-hop question-answering, and these can exhibit poor quality (Chen et al., 2024).

We propose *Source2Synth*, a self-augmentation and curation approach that produces *high quality synthetic data grounded in real-world sources* for two complex tasks. By basing the data generation process on real-world sources, *Source2Synth* steers the examples to be more realistic, diverse, and factually correct. The curation step filters out low-quality data. The importance of data quality in enabling instruction-following capabilities has been demonstrated (Zhou et al., 2023), it is equally important to enable other capabilities in LLMs.

In this paper, we propose *Source2Synth*, a method that generates synthetic data for fine-tuning LLMs, unlocking two advanced capabilities: 1) *question answering with tables*, which requires aggregating information from a table using SQL to answer the question and 2) *multi-hop question answering*, where each question requires finding and reasoning over multiple supporting documents to answer it. Each of these tasks depends on a distinct type of data: the first utilises tables, while the second relies on a collection of documents. Models fine-tuned with *Source2Synth* achieve improved performance without relying on human annotations, providing evidence that this is an effective data generation method.

Source2Synth consists of three stages: *Dataset Generation*, *Dataset Curation*, and *Model Fine-tuning*, see Figure 1. At the *Dataset Generation* stage, we select a data source (such as tables found on the web or articles from Wikipedia) to *ground* the generation in realistic information. Then, our method picks an *attribute or seed* to trigger the generation and condition its components, for instance, an entity in a Wikipedia article or a factual statement about a table. Given the attribute, the method then produces the full example: the question, the reasoning chain (e.g., the steps for multi-hop question answering, or an SQL statement) and an answer.

At the *Data Curation* stage, the augmented dataset is split in two sections: the first half is used to fine-tune a LLM. This intermediate fine-tuned model is used to curate the second section of the synthetic dataset through an imputation step followed by a filtering step using rejection sampling.

For the latter, we reject examples for which the intermediate fine-tuned model fails to produce the correct answer in k tries. This provides a high-quality curated dataset for fine-tuning, which results in a better performing model on a given task.

To summarise, our key contributions are: 1) a novel, scalable approach for producing synthetic data based on real data sources, showcased on two challenging tasks, and 2) a curation method that leverages slice training, filtering, and imputation to produce higher quality data and enhanced task performance.

2 Related Work

Synthetic Data Generation using LLMs A number of works leverage language models to generate synthetic datasets suitable for pre-training, fine-tuning or instruction-tuning. Some rely on knowledge-probing, by generating a continuation or predicting missing words in a close-style template (Schick and Schütze, 2020; Schick and Schütze, 2021; Petroni et al., 2019; Jiang et al., 2019). Other works improve the quality of synthetic data by using different model-based or human filtering techniques (Schick and Schütze, 2021; Liu et al., 2022; Li et al., 2024; Thoppilan et al., 2022; Schimanski et al., 2024). Another line of work consists of generating high-quality synthetic data for general instruction tuning (Yin et al., 2023; Zhou et al., 2023; Wang et al., 2023b; Taori et al., 2023; Honovich et al., 2023; Wang et al., 2022, 2023a) where the LLM generates diverse potential tasks based on the valid fields in the given dataset. However, it is hard to synthetize high-quality multi-hop instruction-tuning data. If a general recipe is used (Wang et al., 2023a), only a small fraction of synthetic instruction-tuning samples are multi-hop, and they can exhibit poor quality (Chen et al., 2024). For obtaining data for tabular question-answering task, previous approaches typically rely on human annotations (Li et al., 2023a).

Source2Synth creates synthetic datasets for two complex tasks: tabular question answering and multi-hop question answering. It leverages real data to identify a task-specific seed, which steers the examples to be more realistic, diverse, and factually correct. Since it uses the LLM itself, it boosts the quality of the data, avoiding human-in-the-loop steps.

Some recent works also leverage real-world documents from the web (Nguyen et al., 2024; Ziegler et al., 2024), open-source code snippets to generate diverse instruction data for code generation (Wei et al., 2024; Dubey et al., 2024) or metadata from existing datasets to create instruction-tuning data (Yin et al., 2023). In contrast, *Source2Synth* does not require a back-translation approach or initial fine-tuning to generate the task-specific seed.

Teaching LLMs to Use Tools Enabling LLMs with tool-use extends their abilities to manipulating structured data, retrieving information from external sources, or interacting with APIs(Parisi et al., 2022; Schick et al., 2023; Tang et al., 2023).

In most approaches, tool usage is restricted to inputs that are strings or numbers. However, a large amount of information is currently stored in relational databases. Unlocking the ability for LLMs to compose SQL queries has unlimited potential, but producing SQL queries that are relevant for a given database schema remains challenging. See Appendix B for an extended related work section.

3 Method

Source2Synth provides a way to generate high-quality synthetic data leveraging self-augmentation and self-improvement using real-world data as sources. The generated synthetic examples can be used for fine-tuning a LLM.

Source2Synth consists of three stages: *Dataset Generation*, *Dataset Curation*, and *Model Fine-tuning*. In particular, we present how to generate synthetic data suitable for two challenging tasks: multi-hop question answering (MHQA), using documents, to test reasoning; and tabular question answering (TQA), using tables, to test tool use.

Multi-hop question answering In MHQA, we generate a synthetic dataset consisting of multi-hop question-answer pairs, augmented with the reasoning chain used to generate the answer. See Figure 2 for an overview of the procedure.

Tabular question answering In TQA, we generate a question-answer dataset where each question is based on a table as the data source. In this case, the synthetic examples are augmented with an SQL statement built from automatically-generated facts based on the initial table. See Figure 3 for an overview of the procedure for TQA.

3.1 Dataset Generation

3.1.1 Data source selection

We first select a data source, which can be an existing dataset re-purposed for the task, a collection of existing data points, or structured information (graphs, tables). There is no need for human annotations on the entries, as *Source2Synth* enriches the data by self-augmentation.

MHQA We use English Wikipedia (Wikipedia contributors, 2004) as the data source, since it contains articles in natural language and additional meta-information such as links to related articles. Firstly, we randomly select an initial article, denoted as D_1, among all available Wikipedia articles. For each article D_1, we collect a pool of $n \geq 2$ related articles, from which we sample a second document D_2.

TQA We use four thousand unlabeled tables from the WikiSQL (Zhong et al., 2017) training dataset as sources.

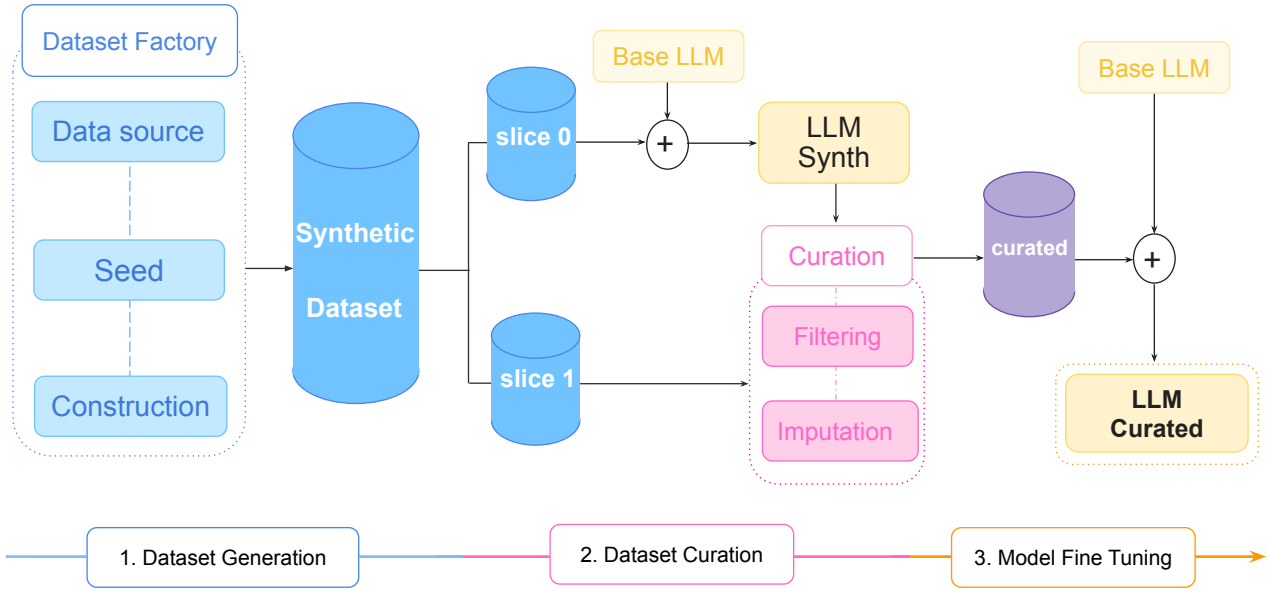

Figure 1 Source2Synth Method. At *Dataset Generation*, we choose a data source to build the dataset. For each example, we select a seed or attribute to condition the generation, and use the data source entry and seed for each example. The resulting synthetic dataset is sliced in two: slice 0, used to fine-tune an intermediate version of the LLM(*LLMSynth*), which is then used to curate slice 1 through filtering and/or imputation during *Dataset Curation* step. At *Model Fine-tuning* stage, the final LLM (*LLMCurated*) is trained on the curated synthetic dataset.

3.1.2 Seed

To create a synthetic example, we first generate a task-specific *seed or attribute* chosen at random from each entry in the source data. The seed anchors the creation of the entry, making it consistent throughout the generation process.

MHQA A *MHQA seed* corresponds to an entity E sampled from the article D_1. E is also the "hop" in the multi-hop question Q, since E links the $n = 2$ sub-questions in which Q is decomposed. For example, in Figure 2, we randomly sample the article D_1 ="The Moon" and the entity E ="Apollo 11", E is the MHQA seed. Then, we pick D_2 ="Neil Armstrong" from the pool of related articles, since it contains a paragraph where "Apollo 11" is included.

TQA A *TQA seed* corresponds to an interesting fact derived from the table when prompting an instruction-tuned language model. For example, in Figure 3, the TQA seed is "The country with most arrivals in 2012". See Figure 11 for the prompt.

3.1.3 Dataset construction

Chain-Of-Thought (Wei et al., 2022) prompting has been used to break a complex task in simpler steps to make it easier to solve by a LLM.

Analogously, we use the task-specific seed to build synthetic step-by-step data, decomposing the generation into intermediate steps.

MHQA We prompt an instruction-tuned language model to generate two questions: a question Q_1 based on D_1, whose answer is the selected entity E; and a second question Q_2, based on D_2 such that its main topic is E. See Figures 16 and 17 for the prompts. For example, in Figure 2, Q_1 = "What was the spaceflight that first landed humans on the Moon?", the hop is E = "Apollo 11" and Q_2 = "Who was the commander of Apollo 11?". We prompt the LLM to merge the two questions to generate the two-hop question Q using the entity as a conceptual link (hop). See Figure 15 for the prompt.

TQA We zero-shot prompt the LLM to generate an SQL statement providing the table and the TQA seed as context, see Figure 12 for the prompt. The generated SQL statement is executed using the *sqlite3*[1] Python library to obtain the answer formatted as a table. If the generated statement is invalid, we discard it.

3.2 Dataset Curation

During curation, the dataset is divided into two sections, each of which contains half the number of synthetic examples. The first section is used to fine-tune an intermediate LLM (denoted by *LLMSynth*). *LLMSynth* is used to self-improve the quality of the second section of the data using an imputation and filtering steps (in purple in Figure 1).

3.2.1 Data filtering

The filtering step consists of using the fine-tuned model *LLMSynth* to predict the output of the question in the synthetic example for k tries. If the output cannot be predicted, it is assumed that the example is low quality and is not included in the final curated dataset.

MHQA and TQA In both cases, we check if the predicted answer given by *LLMSynth* matches the original answer in the synthetically generated example. If after $k = 3$ tries the

[1] https://www.sqlite.org

model has not provided the correct answer, we discard the entry.

3.2.2 Data imputation

The imputation process involves discarding parts of the augmented data points and using *LLMSynth* to fill in the blanks.

MHQA We discard Q_1 and provide *LLMSynth* with Q, Q_2, E, and the sampled document D_1 as context and ask it to reconstruct Q_1. The new candidate Q'_1 for Q_1 is then assessed: if A' (the answer to the new multi-hop question Q' resulting from assembling Q'_1 and Q_2) matches A (the original answer to Q) then we keep the example. We find that asking the model to reconstruct parts of the multi-hop question in-context results in a more natural and cohesive question, removing some of the unnaturalness of the text. In Appendix C.4, we quantify how natural the generated MHQA questions are by measuring perplexity before and after imputation.

TQA the curation process consists only of the filtering step.

In MHQA, the curation step removes around 13% of the originally generated questions. In TQA, we keep 27% of the original examples.

3.3 Model fine-tuning

We can fine-tune a pretrained or an instruction-tuned LLM with the *Source2Synth* synthetic dataset. We use our dataset for supervised training of both the reasoning chain and the final answer. The resulting *LLMCurated* model is equipped with the relevant capability for the task of interest. See Figure 4, for an example response from the model fine-tuned with the *Source2Synth* approach for MHQA and TQA.

4 Experimental Setup

Source2Synth has potential in many important domain-specific applications, such as medical or legal QA, where accessing large quantities of annotated data is costly or impossible due to proprietary constraints. In this paper, for reproducibility purposes, we focus on publicly available data and evaluation benchmarks. Each of these benchmarks contains source data for one of the two tasks of interest: *tabular question answering*, and *multi-hop question answering*. For MHQA, we use the HotpotQA (Yang et al., 2018) benchmark and for TQA, we use the WikiSQL (Zhong et al., 2017) benchmark. We compare our *Source2Synth* method with a number of baselines.

4.1 Multi-Hop QA Experimental Setup

Evaluation data The HotpotQA benchmark (Yang et al., 2018) (HPQA) consists of 113,000 examples of multi-hop question-answer pairs based on Wikipedia, split in train, test, and validation sets. Yang et al. (2018) recommend the FullWiki setup to test multi-hop reasoning abilities of models, for this reason, we use this setup for the evaluation with the test set.

Each entry in HPQA is constructed such that: 1) each question requires finding and reasoning over multiple supporting documents to be able to answer it; 2) each entry provides sentence-level supporting facts for strong supervision and explainability of the prediction; 3) each question is either comparison or bridge type.

A comparison question entails comparing the same concept between n objects, e.g. "Who is the tallest student in class?", while a bridge question builds on a logical and/or causal link and requires deriving statements to get to the answer. E.g. "What is the height of the student that topped the entry exam?" , which requires first identifying the student that topped the exam. The hop length is the number of compared objects, for comparison questions, and for bridge questions, it is the number of links. We choose $n = 2$ to be consistent with HPQA. The test set consists of $7,405$ entries, split evenly between bridge and comparison questions. *Source2Synth* only generates synthetic data for bridge questions. To counterbalance this, we include five hundred comparison questions from HPQA training dataset in the fine-tuning dataset.

Evaluation data contamination checks For each synthetic question generated by *Source2Synth*, we check if its entity E (seed) is present in any of the questions in HPQA's test-set. If so, we check whether the two questions are the same. We found that none of the synthetic data overlaps with the questions in HPQA test set.

Metrics We measure the performance using the soft exact match metric (soft-EM). Soft-EM uses string comparison, it is one if the generated output contains the golden answer and zero otherwise.

Model In the MHQA experiments, we use the Llama2 70B-Chat LLM, we fine-tune it with *Source2Synth* and compare it with other baseline methods. *LLMCurated-datamix* denotes the fine-tuned model that uses *Source2Synth* with 1250 synthetic examples in addition to 500 examples from the HPQA training set. *LLMCurated* denotes the fine-tuned model that uses *Source2Synth* with 1250 synthetic examples only. The 1250 synthetic examples consist of bridge questions only and are generated using a collection of 50 randomly selected Wikipedia articles.

Baselines We compare *Source2Synth* against the following baselines (for all listed models, we use two different prompt templates: a zero-shot and a three-shot CoT, see Figure 14 in Appendix E for details):
Instruction-tuned LLMs: we include both Llama2 70B-Chat and Claude3.5 Sonnet (Anthropic, 2024).
Fine-tuned LLM (HPQA only): we fine-tune Llama2 70B-Chat model with 500 examples from the HPQA training split.
LLMSynth (Synthetic dataset only): we fine-tune Llama2 70B-Chat model with 1250 synthetic examples from Slice 0 (see Figure 1), *without* the data curation step.
LLMSynth-datamix (Synthetic and HPQA): we fine-tune

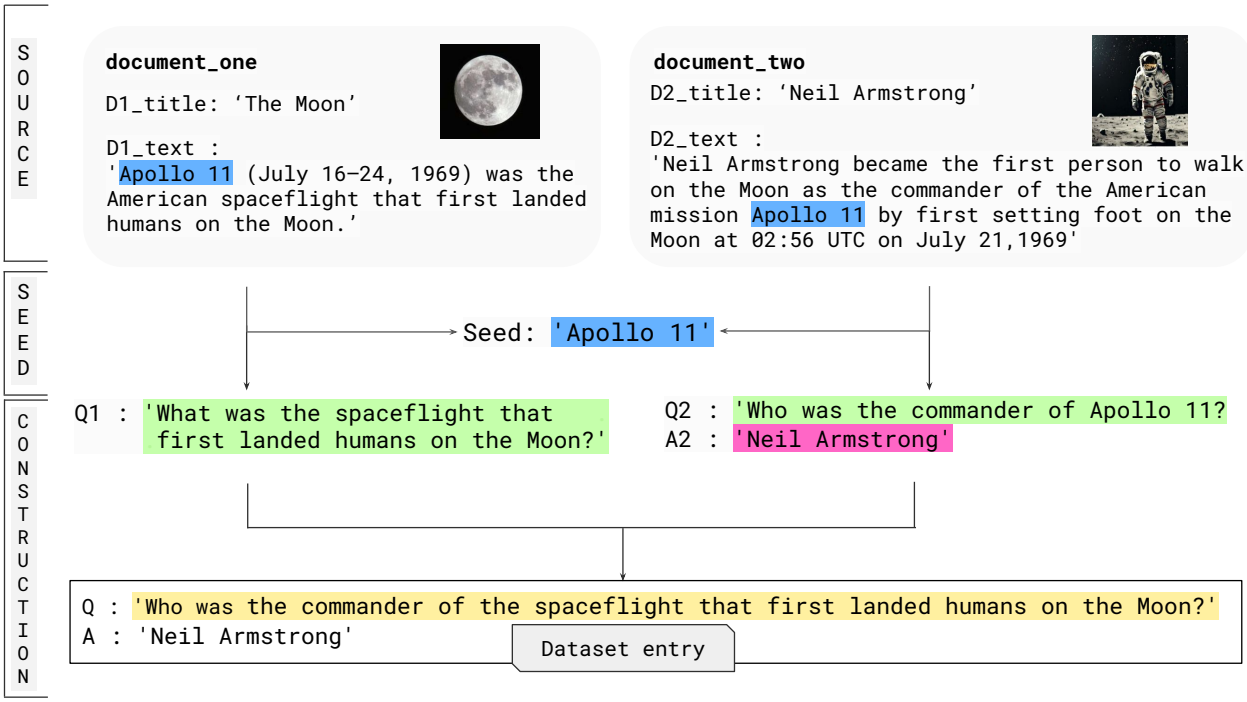

Figure 2 MHQA synthetic data generation process:. We randomly pick an article D_1, e.g. "The Moon". We select the *MHQA Seed* by retrieving an entity E from D_1's entities e.g. "Apollo 11". We then sample from D_1's pool of related articles such that E is present e.g. we select D_2 titled "Neil Armstrong". A question Q_1 is generated from D_1, such that its answer A_1 is the entity E. A second question Q_2 is generated from D_2, such that its main topic is the entity E. We then prompt an LLM to merge the two questions into a multi-hop one Q, based on the common entity. The training example comprises of Q, A, the sub-questions, reasoning chain, and the entity.

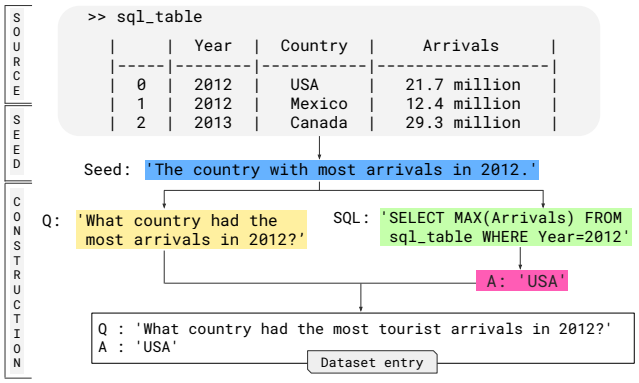

Figure 3 TQA synthetic data generation process. We generate the seed: a fact based on the table. Given the seed and table, an *SQL query* is generated as well as its translation into natural language Q. Then, the SQL is executed to obtain the answer **A**.

Llama2 70B-Chat with the uncurated synthetic data in addition to the 500 HPQA examples.

4.2 Tabular QA Experimental Setup

Evaluation data The WikiSQL benchmark (Zhong et al., 2017) consists of 80,654 hand-annotated examples of natural language questions, SQL queries, and tables created from 24,241 tables extracted from Wikipedia. The validation split contains 7,857 examples after removing non-executable SQL tables, see Appendix D for more details.

Evaluation data contamination checks During the *Source2Synth* synthetic dataset construction, we select the tables in the train split of the WikiSQL benchmark as the source. Since we evaluate with the test set, there is no overlap or evaluation data contamination since these sets are mutually exclusive by design.

Metrics We measure performance using the exact match (EM) and the soft-EM metrics. The EM metric uses string comparison, it equals one if the golden answer is equal to the generated answer and zero otherwise.

Model For TQA, we use the Starchat-beta language model (Li et al., 2023b) as the initial language model (and use the following parameters for finetuning: batch size 32, 100 steps, learning rate of 0.0001 and linear warm-up). The Starchat model is an instruction-tuned LLM with 16 billion parameters trained to act as a helpful coding assistant. This model is a fine-tuned version of StarCoder (Li et al., 2023b), that consists of a LLM pre-trained on a large code corpus, containing SQL statements, and fine-tuned on 35B Python tokens. We generate ten thousand SQL statements based on the source tables and keep eight thousand examples per slice.

Baselines We compare the performance of *Source2Synth* method against a variety of baselines consisting of prompting the Starchat-beta instruction-tuned language model:
Zero-shot Table QA (Figure 7): zero-shot prompt with task instruction, table and question.
One-Shot No Context QA (Figure 8): one-shot example

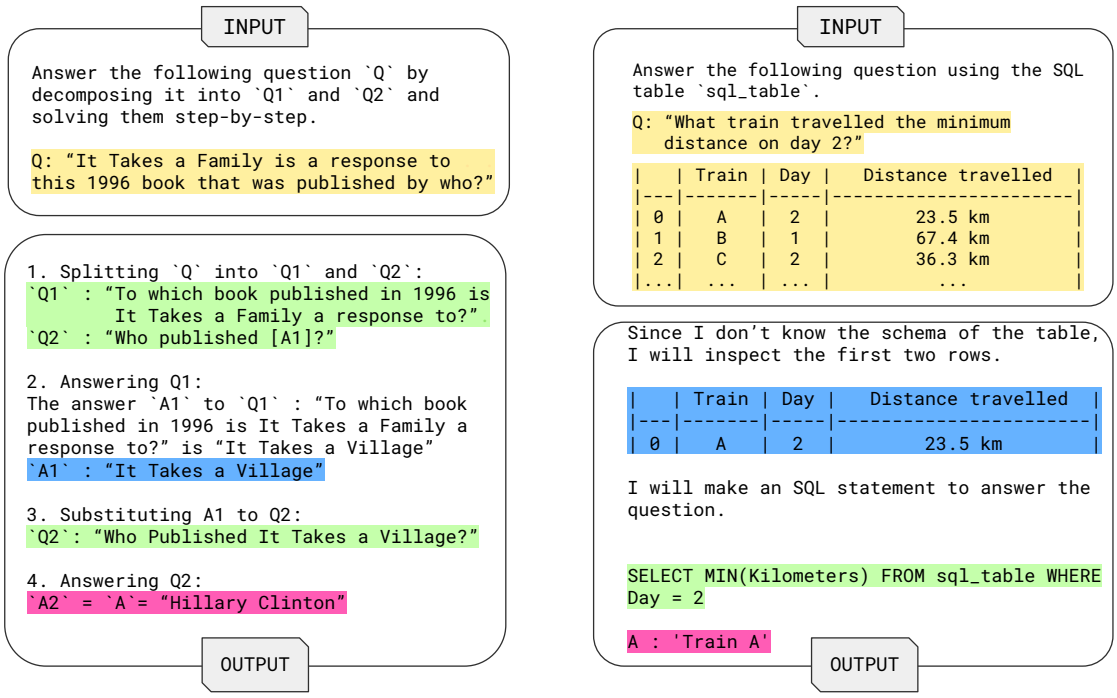

Figure 4 MHQA (left). Model's response to a multi-hop input question (in yellow). The colours correspond to the generation of the augmented entries: the decomposition into sub questions, (in green), the seed (in blue), and the final answer (in pink).
TQA (right). Model's response to a tabular input question (in yellow). The coloured parts correspond to the generation of the augmented entries: SQL (in green), the seed, (in blue), and the final answer (in pink).

with a question and answer prompt with task instruction and the actual question to answer.
One-Shot Table QA (Figure 9): prompt including the table for the one-shot example and the question to answer.
One-shot Table+SQL QA (Figure 10): the prompt includes an example containing the table and question, and an instruction suggesting that the model can leverage an SQL tool. We then execute the predicted SQL to obtain the answer.
LLMSynth: Fine-tune the model with synthetic data *without* applying the data curation step.

5 Results

5.1 Multi-Hop question answering

We report the experimental results in Table 1 using zero-shot and three-shot prompts (Figure 14). All fine-tuned models outperform the first two baselines where we prompt the Llama2 70B-Chat and Claude3.5 Sonnet instruction-tuned models. We observe that using only synthetic data or only HPQA data for fine-tuning has worse performance than when combined, whether or not the synthetic data is curated (*LLMCurated*, *LLMSynth*). In all models where we use the full *Source2Synth* pipeline we see further performance improvements (*LLMCurated*, *LLMCurated-datamix*) vs not curating the data (*LLMSynth*, *LLMSynth-datamix*).

In Table 1, we can simulate the no-data regime by analysing the case where we fine-tune Llama2 70B-Chat with synthetic data only. We observe that fine-tuning without the additional 500 entries from HPQA minimally hinders performance (since the difference between *LLMCurated* vs *LLMCurated-datamix* is 1.13%).

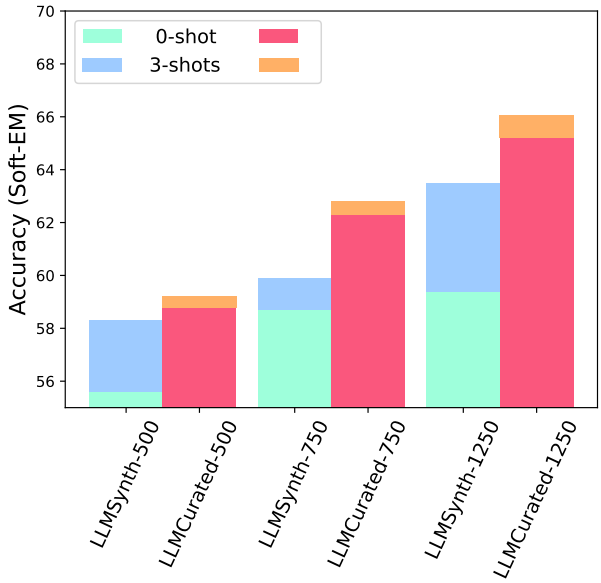

Figure 5 Synthetic Data scaling performance. *Source2Synth* performance when increasing the amount of synthetic data in the data mix, before and after curation. All models in the plot use the 500 HPQA examples but vary the number of synthetic examples, for example, LLMSynth-500 uses 500 HPQA and 500 synthetic examples.

Scaling performance *Source2Synth* can be leveraged when the amount of available data is low. In Figure 5, we study how performance changes when adding more synthetic data

Table 1 Evaluation of Source2Synth on Multi-hop question answering. *LLMCurated-datamix* and *LLMSynth-datamix* are fine-tuned with 500 entries from HPQA and 1250 entries from the *Source2Synth* synthetic data. *LLMSynth* and *LLMCurated* are fine-tuned with 1250 entries from the *Source2Synth* synthetic data only.

Method	0-shot	3-shot CoT prompt
Llama2 70B-Chat	40.45%	44.13%
Claude 3.5 Sonnet	50.3%	53.4%
Fine-tuned LLM (HPQA data only)	53.22%	58.40%
LLMSynth (synthetic data only (bridge questions))	52.31%	56.70%
LLMSynth-datamix (HPQA and synthetic data (bridge questions))	57.46%	62.73%
LLMCurated (synthetic data only (bridge questions))	64.07%	64.68%
LLMCurated-datamix (HPQA and synthetic data (bridge questions))	**65.23%**	**66.05%**

Table 2 Tabular question answering. The models *LLMSynth* and *LLMCurated* are fine-tuned using *Source2Synth* curated synthetic data only. Performance comparison on the WikiSQL test set.

Method	Exact Match	Soft-EM
Starchat-beta (one-shot no context QA)	0.25%	16.22%
Starchat-beta (zero-shot table QA)	1.83%	20.07%
Starchat-beta (one-shot table QA)	2.03%	31.06%
Starchat-beta (one-shot table+SQL tool QA)	12.30%	34.13%
LLMSynth (synthetic data only)	23.86%	34.21%
LLMCurated (synthetic data only)	**34.50%**	**42.80%**

in the fine-tuning data mix, which includes 500 samples from the HPQA train split. We perform the analysis on both *LLMSynth-datamix* and *LLMCurated-datamix* to showcase the impact of the curation technique. During the curation step, the following percentages of samples were removed: 7% for 500, 8% for 750, 11% for 1250.

In both cases, uniformly over all data mix sizes, we see that applying the *Source2Synth* pipeline results in a stronger model. For the *LLMSynth-datamix* model (fine-tuned with uncurated samples), providing more synthetic examples leads to a steady improvement in performance across all data sizes, for both zero-shot and three-shot prompting variants. The *LLMCurated-datamix* model follows a similar trend, consistently outperforming the uncurated version of the model. Overall, we observe that using our synthetic data generation pipeline to construct more synthetic data brings further performance gains.

Analysis of performance with respect to different question types and levels of difficulty In Appendix C.1, we study the capabilities of our model by analysing the performance of LLM-Curated-1250 with a particular focus on the type and difficulty of the questions: hard/medium/easy bridge and comparison questions. In Table 4, we compare the performance of the base model, the model fine-tuned with HPQA only, and the model fine-tuned using *Source2Synth*, according to the difficulty level, provided in the HPQA train dataset. We also subdivide the results according to question-type (bridge vs. comparison).

We observe that *Source2Synth* performs better in all types of questions and difficulties, with an average absolute performance gain of +13.38% on the base LLM and an absolute performance gain of +7.35% compared to the LLM fine-tuned on HPQA. In particular, by applying our method, the resulting model achieves absolute performance gains of +16.8% and +16.5% on hard bridge and comparison questions, respectively, compared to the baseline. Furthermore, it is interesting to see substantial improvement in answering comparison-type questions despite not generating these during dataset construction.

Extending Source2Synth data-generation method to comparison-type questions For comparison questions, we require to compare the same concept between two objects. The dataset generation method from Section 3.1 can be extended to comparison-type questions as follows: For the first document, we select the entities that have named entity recognition (NER) properties such as date, location, nationality, among others. We then select a second document and from its pool of entities, we draw an entity that has the same NER property as the first entity from the first document. We can then prompt a LLM to produce a question that compares the common NER property between both documents.

In Table 3, we show that if we use the *Source2Synth* approach with synthetic data consisting of both bridge and comparison questions, then, *LLMCurated* has comparable performance to *LLMCurated-datamix* from Table 1 which uses comparison questions from the HPQA dataset and synthetically generated bridge questions (1.13% difference).

Effectiveness fine-tuning smaller LLMs (Llama3 8B-instruct and Llama4 17Bx16E) In Appendix C.2, we fine-tune Llama3 8B-instruct and Llama4 17Bx16E on 1250 synthetically-generated examples with *Source2Synth* using Llama2 70B-Chat for the dataset generation. We showcase that our method remains effective when fine-tuning smaller LLMs, resulting in an accuracy increase of 23.06% for the Llama3 8B-instruct LLM and 36.90% for the LLama4 17Bx16E

Table 3 Evaluation of Source2Synth on Multi-hop question answering. The models shown are fine-tuned with 1250 entries from the *Source2Synth* synthetic data for both comparison and bridge questions.

Method	0-shot
LLMSynth (synthetic data (bridge and comparison questions)	56.01%
LLMCurated (synthetic data only (bridge and comparison questions)	**64.5%**

models respectively.

Effect of not using data grounded in real-world sources In Appendix C.3, we apply *Source2Synth* starting from an ungrounded synthetic dataset to show the benefit of real-world data as a source. Namely, if we use ungrounded synthetic data to fine-tune either Llama3 8B-instruct or Llama2 70B-Chat, the resulting models have an accuracy loss of -7% on average.

5.2 Tabular question answering

The experimental results for TQA are shown in Table 2. Providing no context about the table when prompting the instruction-tuned StarChat language model has very poor performance (first row). This is expected, since the questions in WikiSQL require information contained in the table, while the model does not have any other information except for the general knowledge stored in its parameters. Even if we pass the table in the context, the performance does not improve much due to the inherent difficulty to ingest structured data. We observe that passing the table in a zero-shot fashion is not good (second row). Even including an example of table usage in a one-shot fashion (third row) improves only the soft-EM but the EM metric remains low (2.03%). Performance increases once we enable the model to use the SQL tool and provide a one-shot example containing both the relevant table and the SQL query (fourth row, $EM = 12.3\%$).

We observe an increase in performance if we use *Source2Synth* to fine-tune the StarChat model (*LLMCurated*, last row). Indeed, *Source2Synth* performs significantly better than fine-tuning the StarChat language model with uncurated data only (*LLMSynth*, second to last row). Even so, *LLMSynth* still outperforms the other baselines.

6 Conclusion

We introduce *Source2Synth*, a new method for generating high-quality synthetic data for fine-tuning LLMs to unlock two advanced capabilities: 1)*question answering with tables* and 2) *multi-hop question answering*. Our method guides and grounds the generation, taking into account real data sources and filtering for low-quality samples. We believe our method is valuable in domains where unstructured data is available as a source—such as the legal and medical fields—even though this data is typically not readily available in the form of question-answer pairs. Leveraging the *Source2Synth* method bypasses the need to use human annotators to transform such unstructured data into QA pairs. We show that fine-tuning on high quality examples produced with *Source2Synth* consistently leads to substantial improvements while using less data. We see our method as a first step towards building high-quality automatic data generation methods without human input.

References

Anthropic. Claude 3.5 sonnet, 2024.

Gilbert Badaro, Mohammed Saeed, and Paolo Papotti. Transformers for tabular data representation: A survey of models and applications. *Transactions of the Association for Computational Linguistics*, 11:227–249, 03 2023. ISSN 2307-387X. doi: 10.1162/tacl_a_00544. https://doi.org/10.1162/tacl_a_00544.

Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers, 2024. https://arxiv.org/abs/2305.17126.

Zhi Chen, Qiguang Chen, Libo Qin, Qipeng Guo, Haijun Lv, Yicheng Zou, Wanxiang Che, Hang Yan, Kai Chen, and Dahua Lin. What are the essential factors in crafting effective long context multi-hop instruction datasets? insights and best practices, 2024. https://arxiv.org/abs/2409.01893.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022. https://arxiv.org/abs/2204.02311.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional trans-

formers for language understanding, 2019. https://arxiv.org/abs/1810.04805.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaoqing Ellen Tan, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aaron Grattafiori, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alex Vaughan, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Franco, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, Danny Wyatt, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Firat Ozgenel, Francesco Caggioni, Francisco Guzmán, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Govind Thattai, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Karthik Prasad, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kun Huang, Kunal Chawla, Kushal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Maria Tsimpoukelli, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikolay Pavlovich Laptev, Ning Dong, Ning Zhang, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Rohan Maheswari, Russ Howes, Ruty Rinott, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad,

Sumit Gupta, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Kohler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vítor Albiero, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaofang Wang, Xiaojian Wu, Xiaolan Wang, Xide Xia, Xilun Wu, Xinbo Gao, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yuchen Hao, Yundi Qian, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, and Zhiwei Zhao. The llama 3 herd of models, 2024. https://arxiv.org/abs/2407.21783.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models, 2023. https://arxiv.org/abs/2211.10435.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. Chatgpt outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30): e2305016120, 2023. doi: 10.1073/pnas.2305016120. https://www.pnas.org/doi/abs/10.1073/pnas.2305016120.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4320–4333, Online, July 2020a. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.398. https://aclanthology.org/2020.acl-main.398.

Jonathan Herzig, Paweł Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. Tapas: Weakly supervised table parsing via pre-training, 04 2020b.

Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. Unnatural instructions: Tuning language models with (almost) no human labor. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14409–14428, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.806. https://aclanthology.org/2023.acl-long.806/.

Zhengbao Jiang, Frank F. Xu, J. Araki, and Graham Neubig. How can we know what language models know? *Transactions of the Association for Computational Linguistics*, 8:423–438, 2019. https://api.semanticscholar.org/CorpusID:208513249.

Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiaxi Yang, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls, 2023a.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Starcoder: may the source be with you!, 2023b. https://arxiv.org/abs/2305.06161.

Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Omer Levy, Luke Zettlemoyer, Jason Weston, and Mike Lewis. Self-alignment with instruction backtranslation, 2024. https://arxiv.org/abs/2308.06259.

Alisa Liu, Swabha Swayamdipta, Noah A. Smith, and Yejin Choi. WANLI: Worker and AI collaboration for natural language inference dataset creation. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 6826–6847, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-emnlp.508. https://aclanthology.org/2022.findings-emnlp.508.

Ruibo Liu, Jerry Wei, Fangyu Liu, Chenglei Si, Yanzhe Zhang, Jinmeng Rao, Steven Zheng, Daiyi Peng, Diyi Yang, Denny Zhou, and Andrew M. Dai. Best practices and lessons learned on synthetic data, 2024. https://arxiv.org/abs/2404.07503.

Dheeraj Mekala, Jason Weston, Jack Lanchantin, Roberta Raileanu, Maria Lomeli, Jingbo Shang, and Jane Dwivedi-Yu. Toolverifier: Generalization to new tools via self-verification, 2024. https://arxiv.org/abs/2402.14158.

Grégoire Mialon, Roberto Dessi, Maria Lomeli, Christoforos Nalmpantis, Ramakanth Pasunuru, Roberta Raileanu, Baptiste Roziere, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. Augmented language models: a survey. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. https://openreview.net/forum?id=jh7wH2AzKK. Survey Certification.

Thao Nguyen, Jeffrey Li, Sewoong Oh, Ludwig Schmidt, Jason Weston, Luke Zettlemoyer, and Xian Li. Better alignment with instruction back-and-forth translation, 2024. https://arxiv.org/abs/2408.04614.

Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models, 2023. https://arxiv.org/abs/2303.09014.

Aaron Parisi, Yao Zhao, and Noah Fiedel. Talm: Tool augmented language models, 2022. https://arxiv.org/abs/2205.12255.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In Chengqing Zong and Michael Strube, editors, *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natu-*

ral Language Processing (Volume 1: Long Papers), pages 1470–1480, Beijing, China, July 2015. Association for Computational Linguistics. doi: 10.3115/v1/P15-1142. https://aclanthology.org/P15-1142.

Fabio Petroni, Tim Rocktäschel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, Alexander H. Miller, and Sebastian Riedel. Language models as knowledge bases? In *Conference on Empirical Methods in Natural Language Processing*, 2019. https://api.semanticscholar.org/CorpusID:202539551.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su, Huadong Wang, Cheng Qian, Runchu Tian, Kunlun Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, Zhen Zhang, Yining Ye, Bowen Li, Ziwei Tang, Jing Yi, Yuzhang Zhu, Zhenning Dai, Lan Yan, Xin Cong, Yaxi Lu, Weilin Zhao, Yuxiang Huang, Junxi Yan, Xu Han, Xian Sun, Dahai Li, Jason Phang, Cheng Yang, Tongshuang Wu, Heng Ji, Zhiyuan Liu, and Maosong Sun. Tool learning with foundation models, 2023. https://arxiv.org/abs/2304.08354.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding with unsupervised learning. 2018.

Timo Schick and Hinrich Schütze. Few-shot text generation with natural language instructions. In *Conference on Empirical Methods in Natural Language Processing*, 2020. https://api.semanticscholar.org/CorpusID:238260199.

Timo Schick and Hinrich Schütze. Generating datasets with pretrained language models. *ArXiv*, abs/2104.07540, 2021. https://api.semanticscholar.org/CorpusID:233241169.

Timo Schick and Hinrich Schütze. Exploiting cloze questions for few shot text classification and natural language inference, 2021. https://arxiv.org/abs/2001.07676.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. https://openreview.net/forum?id=Yacmpz84TH.

Tobias Schimanski, Jingwei Ni, Mathias Kraus, Elliott Ash, and Markus Leippold. Towards faithful and robust llm specialists for evidence-based question-answering, 2024. https://arxiv.org/abs/2402.08277.

Aneeta Sylolypavan, Derek Sleeman, Honghan Wu, and Malcolm Sim. The impact of inconsistent human annotations on ai driven clinical decision making. *npj Digital Medicine*, 6(1):26, 2023. doi: 10.1038/s41746-023-00773-3. https://doi.org/10.1038/s41746-023-00773-3.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *ArXiv*, abs/2306.05301, 2023. https://api.semanticscholar.org/CorpusID:259108190.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, YaGuang Li, Hongrae Lee, Huaixiu Steven Zheng, Amin Ghafouri, Marcelo Menegali, Yanping Huang, Maxim Krikun, Dmitry Lepikhin, James Qin, Dehao Chen, Yuanzhong Xu, Zhifeng Chen, Adam Roberts, Maarten Bosma, Vincent Zhao, Yanqi Zhou, Chung-Ching Chang, Igor Krivokon, Will Rusch, Marc Pickett, Pranesh Srinivasan, Laichee Man, Kathleen Meier-Hellstern, Meredith Ringel Morris, Tulsee Doshi, Renelito Delos Santos, Toju Duke, Johnny Soraker, Ben Zevenbergen, Vinodkumar Prabhakaran, Mark Diaz, Ben Hutchinson, Kristen Olson, Alejandra Molina, Erin Hoffman-John, Josh Lee, Lora Aroyo, Ravi Rajakumar, Alena Butryna, Matthew Lamm, Viktoriya Kuzmina, Joe Fenton, Aaron Cohen, Rachel Bernstein, Ray Kurzweil, Blaise Aguera-Arcas, Claire Cui, Marian Croak, Ed Chi, and Quoc Le. Lamda: Language models for dialog applications, 2022. https://arxiv.org/abs/2201.08239.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. https://arxiv.org/abs/2307.09288.

Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Neural Information Processing Systems*, 2017. https://api.semanticscholar.org/CorpusID:13756489.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, Eshaan Pathak, Giannis Karamanolakis, Haizhi Lai, Ishan Purohit, Ishani Mondal, Jacob Anderson, Kirby Kuznia, Krima Doshi, Kuntal Kumar Pal, Maitreya Patel, Mehrad Moradshahi, Mihir Parmar, Mirali Purohit, Neeraj Varshney, Phani Rohitha Kaza, Pulkit Verma, Ravsehaj Singh Puri, Rushang Karia, Savan Doshi, Shailaja Keyur Sampat, Siddhartha Mishra, Sujan Reddy A, Sumanta Patro, Tanay Dixit, and Xudong Shen. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5085–5109, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.340. https://aclanthology.org/2022.emnlp-main.340/.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu,

Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508, Toronto, Canada, July 2023a. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.754. https://aclanthology.org/2023.acl-long.754/.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions, 2023b. https://arxiv.org/abs/2212.10560.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *CoRR*, abs/2201.11903, 2022. https://arxiv.org/abs/2201.11903.

Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with oss-instruct, 2024. https://arxiv.org/abs/2312.02120.

Wikipedia contributors. Plagiarism — Wikipedia, the free encyclopedia, 2004. https://en.wikipedia.org/w/index.php?title=Plagiarism&oldid=5139350. [Online; accessed 22-July-2004].

Wenhan Xiong, Xiang Li, Srini Iyer, Jingfei Du, Patrick Lewis, William Wang, Yashar Mehdad, Wen-tau Yih, Sebastian Riedel, Douwe Kiela, and Barlas Oğuz. Answering complex open-domain questions with multi-hop dense retrieval, 09 2020.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing*, 2018. https://api.semanticscholar.org/CorpusID:52822214.

Da Yin, Xiao Liu, Fan Yin, Ming Zhong, Hritik Bansal, Jiawei Han, and Kai-Wei Chang. Dynosaur: A dynamic growth paradigm for instruction-tuning data curation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4031–4047, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.245. https://aclanthology.org/2023.emnlp-main.245/.

Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. Tabert: Pretraining for joint understanding of textual and tabular data, 05 2020.

Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103, 2017.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. Lima: Less is more for alignment, 2023. https://arxiv.org/abs/2305.11206.

Ingo Ziegler, Abdullatif Köksal, Desmond Elliott, and Hinrich Schutze. Craft your dataset: Task-specific synthetic dataset generation through corpus retrieval and augmentation. *ArXiv*, abs/2409.02098, 2024. https://api.semanticscholar.org/CorpusID:272367350.

A Limitations

- **Two hops for MHQA** The MHQA number of hops is restricted to two in this paper. However, *Source2Synth* can be extended to a number of hops greater than two. This can be done by looping the dataset generation steps and feeding the result of the previous step as input to the next one.
- **There exist a single table per query to obtain the answer in TQA** We use a single table per query in TQA. However, *Source2Synth* can be extended to more complex scenarios that require to first identify the relevant table from a set of tables. In order to do this, we could use table retrieval and leverage Herzig et al. (2020b) as an encoder model for tables.
- **Multi-table tool use is not supported** *Source2Synth* cannot handle queries that require to aggregate information contained in multiple tables e.g. SQL join statements.
- **Use of rejection sampling for MHQA dataset construction** Our method could be improved with more clever sampling techniques beyond rejection sampling, for instance, leveraging the multihop retrieval approach of Xiong et al. (2020).
- **Source2Synth is restricted to question-answering tasks** We focus on two different data-types as sources, documents and tables, hence, *Source2Synth* can be extended to any domain that has such data-types as source even if it is not publicly available such as: legal and medical domains, among others. We only tackle question-answering tasks and focus on teaching the model the skills of producing reasoning chains in the MHQA format or producing SQL statements relevant to a given (source) table. We consider extending our method to other tasks and skills an interesting future direction.
- **Source data needs to be checked for inconsistencies** We assume that if the user uses a given data source, some initial checks are conducted to remove inconsistencies, such as checking if the tables are complete or if the webpages used are duplicated or contain misinformation. However, some inconsistencies in the data source are potentially mitigated by the filtering process, since the answerability check helps with logical soundness. In the TQA case, we discard nonexecutable SQL statements, but the tables need to be properly parsed by the sqlite library.

B Extended related work

Teaching LLMs to use tools Enabling LLMs with tool use extends their abilities to manipulating structured data, retrieving information from external sources, or interacting with APIs. Various works augment LLMs with general tools or API calls (Parisi et al., 2022; Schick et al., 2023; Tang et al., 2023), possibly interleaving reasoning steps with API calls (Gao et al., 2023; Cai et al., 2024; Paranjape et al., 2023). Some works investigate the use of unseen tools at test time (Paranjape et al., 2023; Mekala et al., 2024). Mialon et al. (2023) and Qin et al. (2023) provide an in-depth review of research on augmented language models.

In most approaches, tool usage is restricted to inputs that are strings or numbers. However, a large amount of information is currently stored in relational databases. Unlocking the ability to compose SQL queries has unlimited potential, but producing SQL queries that are relevant for a given database schema remains challenging.

SQL for LLM tool-usage and transformers for tabular data A variety of benchmarks (Pasupat and Liang, 2015) have been proposed to evaluate the ability of the LLM to generate relevant SQL and their performance in tabular-based question answering (Li et al., 2023a; Zhong et al., 2017).

Alternatively, other works adapt language models to directly handle tabular data (Herzig et al., 2020a; Yin et al., 2020). Badaro et al. (2023) provide a comprehensive overview of works that modify the transformer architecture for tabular data.

C Further experiments

C.1 Analysis of performance on different question types and levels of difficulty

We study the capabilities of our model by analysing the performance of LLM-Curated-1250 with a particular focus on the type and difficulty of the questions: hard/medium/easy bridge and comparison questions. In Table 4, we compare the performance of the base model, the model fine-tuned only on HPQA, and the model fine-tuned using *Source2Synth*, according to the difficulty level provided in the HPQA train dataset. We also subdivide the results according to the type of question (bridge vs. comparison).

Table 4 Analysis of MHQA bridge and comparison questions with respect to level of difficulty. We evaluate models on the full HPQA train dataset (where questions are labelled with easy, medium and hard). *Source2Synth* outperforms the baseline and the fine-tuned on HotpotQA model, yielding a LLM capable of handling hard questions of both types using 1250 synthetic examples.

	Bridge			*Comparison*		
Model	**Hard**	**Medium**	**Easy**	**Hard**	**Medium**	**Easy**
Llama2 70B-Chat	14.5%	27.2%	30.1%	66.6%	71.3%	73.2%
Fine-tuned LLM (HPQA data only)	20.1%	29.8%	34.3%	74.5%	78.3%	82.1%
LLMCurated (Synthetic data only)	27.6%	32.3%	36.2%	79.1%	82.3%	88%
LLMCurated-datamix (Synthetic and HPQA)	**31.3%**	**35.6%**	**39.7%**	**83.1%**	**85.7%**	**87.8%**

C.2 Non-monolithic setting: fine-tuning smaller LLMs using a different model for data generation

We fine-tune Llama3 8B-instruct and Llama4 17Bx16E on 1250 synthetically generated examples resulting from the *Source2Synth* pipeline and on 500 entries from HotpotQA. In both cases, we use Llama2 70B-Chat for dataset generation. For Llama3 8B-instruct we only generate synthetic bridge questions whereas for Llama4, we generate both synthetic bridge and comparison questions. For evaluation, we use the 0-shot prompt from Figure 14 and the soft-EM as a metric. In both cases, *Source2Synth* gives a performance boost with respect to the corresponding base LLMs. For Llama3 8B-instruct, in Table 5, we can see that compared to the performance of the base model, *LLMCurated* shows an increase in accuracy of 23.06%. For Llama4 17B 17Bx16E, in Table 6, we can see that compared to the performance of the base model, *LLMCurated* shows an increase in accuracy of 36.90%.

Table 5 Performance of *Source2Synth* fine-tuning Llama3 8B-instruct using Llama2 70B-Chat for dataset generation

Model	**0-shot**
Llama3 8B-instruct	57.8%
LLMSynth (synthetic data only (bridge questions))	64.46%
LLMCurated (synthetic data only (bridge questions))	**71.13%**

Table 6 Performance of *Source2Synth*fine-tuning Llama4-17Bx16E using Llama2 70B-Chat for dataset generation

Model	**0-shot**
Llama4-17Bx16E base LLM	49.6%
LLMSynth (synthetic data only (bridge and comparison questions))	58.7%
LLMCurated (synthetic data only (bridge and comparison questions))	**67.9%**

C.3 Applying Source2Synth on made up toy data only

We tried *Source2Synth* starting with a fully made up toy dataset (ungrounded data means no real-world source), consisting of 1250 synthetic data points for fine-tuning plus 500 entries from HotpotQA. To ensure diversity in the generation, we ask the model to generate a question based on two topics A and B picked from the following list: ["Moon", "Ocean", "Water", "Wolf", "Tides", "Day", "Light", "Apple", "United States", "Europe", "Roman Empire", "Chocolate", "Environment", "India", "Strawberries", "Physics", "Pen", "Sugar", "History", "Jelly", "Mug", "Cat", "Lion", "Flower", "Purple", "Red", "Stars", "Electricity", "Paper", "Snow", "Mount Everest", "Table", "Friendship", "Book", "Laptop", "Phone", "Mushroom", "Hat", "Coffee", "Pasta", "Island", "Volcano", "Storm", "Key", "Candle", "Asia", "Desert", "Tree", "River"]

The resulting two-hop questions are bridge-type and have lower perplexity than those generated starting from a grounded source. The model trained on ungrounded samples performs worse than the one trained on grounded ones. Comparing Table 7 to Table 5, we can see that the loss in accuracy in the case of Llama3 8B-instruct is -7.17%. For Llama2 70B-Chat, comparing Table 8 with Table 1 the accuracy loss is -6.82%. We also observe that there are repeating patterns in the structure of the questions generated, we hypothesize that this hinders generalization. For example, many of the synthetically ungrounded questions follow this structure: 'What / Who [Q1] and / or What [Q2]?' " (i.e. "What is the name of the ancient mythological figure that is often depicted as being able to transform into a wolf, and is also associated with the full moon that occurs in March, which is also known as the Worm Moon?").

Table 7 Performance of *Source2Synth* using made up (ungrounded) toy data as source, fine-tuning Llama3 8B-instruct, using Llama2-70B Chat for dataset generation.

Model	Accuracy
Llama3 8B-instruct	57.80%
LLMSynth (ungrounded toy data only)	60.45%
LLMCurated (ungrounded toy data only)	**66.37%**

Table 8 Performance of *Source2Synth* using made up (ungrounded) toy data as source,fine-tuning Llama2 70B-Chat, using Llama2 70B-Chat for dataset generation.

Model	Accuracy
LLama2 70B-Chat	40.45%
LLMSynth (ungrounded toy data only)	51.90%
LLMCurated (ungrounded toy data only)	**59.70%**

C.4 On the impact of imputation

We studied the perplexity of questions before and after imputation for 1) synthetic data generated from a grounded source like Wikipedia and 2) for synthetic ungrounded data. In both cases, the imputation step lowers perplexity and reduces the unnaturalness of the question. In Table 9 we report the average perplexity score and in Figure 6 we showcase an example of how imputation leads to rephrasing sentences in a more natural way.

Table 9 Average perplexity of generated questions before and after imputation

Model	PPL before imputation	PPL after imputation
Synthetic grounded data	24.7	13.6
Synthetic ungrounded data	15.51	8.33

C.5 More results on prompt engineering

Table 10 MHQA using different prompts. Llama2 70B-Chat accuracy across different prompts.The prompt for *Role* is "You are a QA-robot. Answer the following question:".

Prompt Type	Model Accuracy (soft-EM, hotpotQA test set)
0-shot	40.45%
Role	22.34%
1-shot	26.65%
Few-shots (5-shots)	21.83%
Role (1-shot)	28.29%

D SQL non-executable code filtering

We discard SQL statements which cannot be executed with *sqlite3*[2]. Out of 50 tables, we generate 800 seed statements and keep 658 executable SQL statements.

E Prompts

[2]https://www.sqlite.org

Comparing Q pre- and post- imputation

Before imputation:
Q: "What pet did the poet and father of mathematician Ada Lovelace had when he was a student at Trinity out of resentment for rules forbidding pet dogs like his beloved Boatswain?"
Q_1: "What pet did the poet Lord Byron had when he was a student at Trinity out of resentment for rules forbidding pet dogs like his beloved Boatswain?"
Q_2: "Who is the father of mathematician Ada Lovelace?"
E: "Lord Byron"
A: "A bear"
D_1: "Lord Byron also kept a tame bear while he was a student at Trinity out of resentment for rules forbidding pet dogs like his beloved Boatswain."

After imputation:
Q': "What pet did the poet and father of mathematician Ada Lovelace had when he was a student at Trinity?"
Q'_1 : "What pet did the poet Lord Byron had when he was a student at Trinity?"

Figure 6 Comparing Q pre- and post- imputation

Zero-shot Table QA prompt.

Answer the following question using the table below. You may leverage an SQL tool.

{table}

Q: {question}

Figure 7 Zero-shot Table QA prompt for the TQA task.

One-Shot No context QA prompt.

– Example –
Q: What was the last year where this team was part of the US A-league?
A: 2004

Now do the same for the following question.
Q: {question}

Figure 8 One-Shot No context QA prompt for the TQA task.

One-shot Table QA prompt.

-- Example --
Answer the following question using the table below.
Your answer should be short and concise.

```
Season | Team       | League_apps | Goals
1923   |Swindon Town | 55         | 3
1922   |Swindon Town | 14         | 4
1921   |Swindon Town | 24         | 11
1920   |Swindon Town | 26         | 16
1919   |Swindon Town | 20         | 10
1914   |Swindon Town | 23         | 12
1913   |Swindon Town | 24         | 18
1912   |Swindon Town | 12         | 9
1911   |Swindon Town | 20         | 16
1910   |Swindon Town | 30         | 19
1909   |Swindon Town | 33         | 19
1908   |Swindon Town | 34         | 28
1907   |Swindon Town | 30         | 17
```

Q: How many league appearances were there between 1907 and 1909 (inclusive)?
A: 97

Now do the same for the following table and question.

{table}

Q: {question}

Figure 9 One-shot Table QA prompt for the TQA task.

One-shot Table+SQL QA prompt.

-- Example --
Answer the following question using the table below.
You may leverage an SQL tool.
The table is stored in a variable 'sql_table' and has the following schema:

```
Season | Team        | League_apps | Goals
1923   |Swindon Town | 55          | 3
1922   |Swindon Town | 14          | 4
```

Q: How many league appearances were there between 1907 and 1909 (inclusive)?

```
SQL: SELECT SUM(League_apps) FROM sql_table WHERE Season BETWEEN 1907 AND 1909
     | Result
result | 97
```

Now do the same for the following table and question.

{table}

Q: {question}

Figure 10 One-shot Table+SQL QA prompt for the TQA task.

Generating a seed in TQA.

Please generate an interesting statement about this table. The statement is a fact about one of the columns in the following table.
{table}

An interesting statement as a result of this is:

Figure 11 Prompt used to induce a pertinent and interesting seed topic in TQA. This is done zero-shot.

Generating meaningful SQL in TQA.

Please generate SQL statements for the following table:

{table}

Seed: {seed}

An interesting SQL statement as a result of this is

Figure 12 Prompt used to induce a meaningful SQL statement given the table and seed for the TQA task. This is done zero-shot.

Generating a question in TQA.

I want to convert an SQL statement into a question.
Here is the original table:
{table}

SQL: {SQL}

What is the question that this SQL statement would be the answer to?

Figure 13 Prompt used to induce a meaningful question using the table and generated SQL query for the TQA task. This is done zero-shot.

Three-shot CoT prompt used at evaluation time on MHQA.

Answer the following multi-hop question ‘Q’ by decomposing it into ‘Q1’ and ‘Q2’ and solving them step-by-step. Learn from the following 3 examples. As shown in the following example:

-- Example #1 --
‘Q’ = ‘Who was the commander of the spaceflight that first landed humans on the Moon?’

1. Splitting ‘Q’ into ‘Q1’ and ‘Q2’:
‘Q1’ : ‘What was the spaceflight that first landed humans on the Moon?’;
‘Q2’ : ‘Who was the commander of [A1]?’;

2. Answering Q1:
The answer ‘A1’ to ‘Q1’ : ‘What was the spaceflight that first landed humans on the Moon?’ is ‘Apollo 11’. ‘A1’ = ‘Apollo 11’

3. Substituting A1 to Q2:
‘Q2’ : ‘Who was the commander of Apollo 11?’,

4. Answers Q2:
The answer ‘A2’ to Q2’ : ‘Who was the commander of Apollo 11?’ is ‘Neil Armstrong’.
‘A2’ = ‘A’ = ‘Neil Armstrong’

-- Example #2 --
‘Q’ = ‘What is the main ingredient in the flagship product of Ferrero?’

1. Splitting ‘Q’ into ‘Q1’ and ‘Q2’:
‘Q1’: ‘What is the flagship product of Ferrero?’
‘Q2’: ‘What is the main ingredient in [A1]?’

2. Answering Q1:
The answer ‘A1’ to ‘Q1’ : ‘What is the flagship product of Ferrero?’ is Nutella’.‘A1’ = Nutella’

3. Substituting A1 to Q2:
‘Q2’ : ‘What is the main ingredient in Nutella?’,

4. Answers Q2:
The answer ‘A2’ to Q2’ : ‘What is the main ingredient in Nutella?’.
‘A2’ = ‘A’ = ‘Hazelnuts

--Example #3 --

‘Q’ = ‘Who was the Roman Emperor when Jesus was born?’
1. Splitting ‘Q’ into ‘Q1’ and ‘Q2’:
‘Q1’: ‘When was Jesus born? ‘
‘Q2’: ‘Who was the Roman Emperor in [A1]?’

2. Answering Q1:
The answer ‘A1’ to ‘Q1’ : ‘When was Jesus born?’ is 1 BCE. ‘A1’ = 1 BCE

3. Substituting A1 to Q2:
‘Q2’ : ‘Who was the Roman Emperor in 1 BCE?’,

4. Answers Q2:
The answer ‘A2’ to Q2’ : ‘Who was the Roman Emperor in 1 BCE?’.
‘A2‘ = ‘A‘ = ‘Caesar Augustus‘

You MUST apply this structure when asked to answer a multi-hop question ‘Q’. Now answer the multi-hop question ‘Q‘ as shown in the examples above.
Q: {question}

Figure 14 *Three-shot CoT prompt* used at evaluation time in MHQA.

Prompt used to merge Q1 and Q2 in MHQA.

Merge ‘Q1‘ and ‘Q2’ into a single multi-hop bridge question ‘Q’.
Learn from the following 3 examples. As shown in the following example:

-- Example #1 --

‘Q1’ : "What was the spaceflight that first landed humans on the Moon?”
‘Q2’: "Who was the commander of Apollo 11?”

Solution:
1. Answer Q1; ‘A1’ is "Apollo 11”
2. If ‘A1’ is in ‘Q2’ print(A1); ‘A1’ = Apollo 11 is in ‘Q2’ so I print "Apollo 11”
3. Since you found ‘A1’ in ‘Q2’, rewrite ‘Q2’ so that you delete ‘A1’ and substitute ‘Q1’ there;
Rewriting Q2. Original ‘Q2’: "Who was the commander of Apollo 11?”. Since ‘A1’ is in ‘Q2’, I delete it and write ‘Q1’ there. Rewritten ‘Q2’: "Who was the commander of the spaceflight that first landed humans on the Moon?”

The single multi-hop question is therefore the rewritten ‘Q2’.
‘Q2‘ = ‘Q‘ = "Who was the commander of the spaceflight that first landed humans on the Moon?”

-- Example #2 --

‘Q1’: What is the flagship product of Ferrero?
‘Q2’: What is the main ingredient in Nutella?
Solution:
1. Answer Q1; ‘A1’ is "Nutella”
2. If ‘A1’ is in ‘Q2’ print(A1); ‘A1’ = "Nutella” is in ‘Q2’ so I print "Nutella”
3. Since you found ‘A1’ in ‘Q2’, rewrite ‘Q2’ so that you delete ‘A1’ and substitute ‘Q1’ there;
Rewriting Q2. Original ‘Q2’: "What is the main ingredient in Nutella?”.
Since ‘A1’ is in ‘Q2’, I delete it and write ‘Q1’ there.
Rewritten ‘Q2’: "What is the main ingredient in the flagship product of Ferrero?”

The single multi-hop question is therefore the rewritten ‘Q2’. ‘Q2’ = ‘Q’ = "What is the main ingredient in the flagship product of Ferrero?”

-- Example #3 --

‘Q1’: "When was Jesus born?”
‘Q2’: "Who was the Roman Emperor in 1 BCE?”

Solution:
1. Answer Q1; ‘A1’ is "1 BCE”
2. If ‘A1’ is in ‘Q2’ print(A1); ‘A1’ = 1 BCE is in ‘Q2’ so I print “1 BCE”
3. Since you found ‘A1’ in ‘Q2’, rewrite ‘Q2’ so that you delete ‘A1’ and substitute ‘Q1’ there;
Rewriting Q2. Original ‘Q2’: "Who was the Roman Emperor in 1 BCE?”. Since ‘A1’ is in ‘Q2’, I delete it and write ‘Q1’ there. Rewritten ‘Q2’: "Who was the Roman Emperor when Jesus was born?"

The single multi-hop question is therefore the rewritten ‘Q2’.
‘Q2’ = ‘Q’ = "Who was the Roman Emperor when Jesus was born?”

You MUST apply this structure when asked to merge ‘Q1’ and ‘Q2’.
Now merge ‘Q1’ and ‘Q2’ into a single multi-hop bridge question ‘Q’.
‘Q2’ : {question1}
‘Q2’ : {question2}

Figure 15 Prompt used to merge Q1 and Q2 in MHQA.

Generating Q1 in MHQA.

Identify one entity in the following text. Come up with a question so that the answer to this question is the entity chosen earlier. The question must be based on the following text. Write your results as ’Question:’ and then the question and ’Entity:’ and then the entity.

Text: {document_one}

Figure 16 Prompt used to generate Q_1. Q_1 is generated such that its answer $A1 = E$ where E is the entity retrieved.

Generating Q2 in MHQA.

Come up with a question based on the following text that contains the word:
{entity}

Text: {document_two}

Figure 17 Prompt used to generate Q_2. Q_2 is generated such that its main topicis E where E is the entity retrieved.